The Porter Stemming Algorithm

- A **vowel** is one of the letters *a, e, i, o* and *u*.
- All other letters excluding *y* are **consonants**.
- The letter *y* is a **consonant** if the preceding letter is *a, e, i, o, u*, or if it starts the word; Otherwise, *y* is a **vowel**.

Examples:

toy contains the consonants *t* and *y*. *o* is a vowel.

syzygy contains the consonants *s, z* and *g*. Here, each occurrence of *y* is a vowel.

yym contains the consonant *y*, the vowel *y* and the consonant *m*, in that order.

- A single consonant is denoted by c and a single vowel by v.
- A list ccc... of length greater than 0 is denoted by C.
- A list vvv... of length greater than 0 is denoted by V.
- Any word, or part of a word, has one of four forms:

CVCV...C
CVCV...V
VCVC...C
VCVC...V

- These can be denoted as:

[C] VCVC... [V]

where [C] means an optional C and [V] means an optional V.

- The above expression can be re-written as:

[C] $(VC)^m$ [V]

- *m* is called the *measure* of the word or word part.

Examples:

m=0 *tr, ee, tree, y, by*

tr is a C i.e. C () in the above expression.
ee is a V i.e. () V.
tree is a C *tr* followed by a V *ee*,
i.e. C () V.
y is a C i.e. C ().
by is a C *b* followed by a V *y*,
i.e. C () V.

m=1 *trouble, oats, trees, ivy*

tr-ou-bl-e is C (V C) V.
oa-ts is (VC).
tr-ee-s is C (VC).
i-v-y is (V C) V.

m=2 *troubles, private, oaten, orrery*

tr-ou-bl-e-s is C (VCVC).
pr-i-v-a-t-e is C (VCVC) V.
oa-t-e-n is (VCVC)
o-rr-e-r-y is (VCVC) V

- Suffix stripping is carried out using a series of rules.
- Each rule is of the form:

(condition) S1 → S2

- This means 'if a word ends with suffix S1 and the stem *before* S1 satisfies the condition, replace S1 with S2. S2 can be null, in which case S1 is simply removed.
- The condition can take the following forms:

(m > *n*) For some integer n. Means that the condition is satisfied if the stem before S1 has a measure greater than *n*.

(m = *n*) The condition is satsified if m equals *n*.

*S	The stem ends in *s*. Similarly for other letters.
*v	The stem ends in a vowel.
*d	The stem ends in a double consonant, e.g. *tt*, *ss*.
*o	The stem ends cvc where the second c is not *w*, *x*, or *y*. (e.g. *wil, hop*).

- The condition may also contain boolean expressions using *and*, *or* and *not*.

Example conditions:

(m=1)	The measure of the stem before S1 is 1.
(m>1 and (*S or *T))	The measure of the stem before S1 is greater than 1 and the stem ends in *s* or *t*
(*d and not (*L or *S or *Z))	The stem before S1 ends in a double consonant other than *l*, *s* or *z*.

- In a set of rules written as a group, if several rules match, the rule with the longest matching S1 is used.

For example, with the following rules (with no conditions):

SSES → SS
IES → I
SS → SS
S →

caresses maps to *caress* since *sses* is the longest match for S1. Thus the last rule is not used even though it matches.

caress maps to *caress* (S1=*ss*) even though the last rule also matches.

cares maps to *care* (S1=*s*).

Step 1a of Algorithm

SSES	→	SS	*caresses*	→	*caress*
IES	→	I	*ponies*	→	*poni*
			ties	→	*ti*
SS	→	SS	*caress*	→	*caress*
S	→		*cats*	→	*cat*

Step 1b Part I

(m>0)	EED	→	EE	*feed*	→	*feed*
				agreed	→	*agree*
(*v*)	ED	→		*plastered*	→	*plaster*
				bled	→	*bled*
(*v*)	ING	→		*motoring*	→	*motor*
				sing	→	*sing*

Step 1b Part II

If the second or third rules in Step 1b is successful, the following rules are applied:

AT	→	ATE	*conflat(ed)*	→	*conflate*
BL	→	BLE	*troubl(ing)*	→	*trouble*
IZ	→	IZE	*siz(ed)*	→	*size*
(*d and not (*L or *S or *Z))	→	single letter	*hopp(ing)*	→	*hop*
			tann(ed)	→	*tan*
			fall(ing)	→	*fall*
			hiss(ing)	→	*hiss*
			fizz(ed)	→	*fizz*
(m=1 and *o)	→	E	*fail(ing)*	→	*fail*
			fil(ing)	→	*file*

Step 1c

(*v*) Y	→	I	*happy*	→	*happi*
			sky	→	*sky*

Step 2

(m>0)	ATIONAL	→	ATE	*relational*	→	*relat*
(m>0)	TIONAL	→	TION	*conditional*	→	*condition*
				rational	→	*rational*
(m>0)	ENCI	→	ENCE	*valenci*	→	*valence*
(m>0)	ANCI	→	ANCE	*hesitanci*	→	*hesitance*
(m>0)	IZER	→	IZE	*digitizer*	→	*digitize*
(m>0)	ABLI	→	ABLE	*conformabli*	→	*conformable*
(m>0)	ALLI	→	AL	*radicalli*	→	*radical*
(m>0)	ENTLI	→	ENT	*differentli*	→	*different*
(m>0)	ELI	→	E	*vileli*	→	*vile*
(m>0)	OUSLI	→	OUS	*analogousli*	→	*analogous*
(m>0)	IZATION	→	IZE	*vietnamization*	→	*vietnamize*
(m>0)	ATION	→	ATE	*predication*	→	*predicate*
(m>0)	ATOR	→	ATE	*operator*	→	*operate*
(m>0)	ALISM	→	AL	*feudalism*	→	*feudal*
(m>0)	IVENESS	→	IVE	*decisiveness*	→	*decisive*
(m>0)	FULNESS	→	FUL	*hopefulness*	→	*hopeful*
(m>0)	OUSNESS	→	OUS	*callousness*	→	*callous*
(m>0)	ALITI	→	AL	*formaliti*	→	*formal*
(m>0)	IVITI	→	IVE	*sensitiviti*	→	*sensitive*
(m>0)	BILITI	→	BLE	*sensibiliti*	→	*sensible*

Step 3

(m>0)	ICATE	→	IC	*triplicate*	→	*triplic*
(m>0)	ATIVE	→		*formative*	→	*form*
(m>0)	ALIZE	→	AL	*formalize*	→	*formal*
(m>0)	ICITI	→	IC	*electriciti*	→	*electric*
(m>0)	ICAL	→	IC	*electrical*	→	*electric*
(m>0)	FUL	→		*hopeful*	→	*hope*
(m>0)	NESS	→		*goodness*	→	*good*

Step 4

(m>1)	AL	→	*revival*	→	*reviv*
(m>1)	ANCE	→	*allowance*	→	*allow*
(m>1)	ENCE	→	*inference*	→	*infer*
(m>1)	ER	→	*airliner*	→	*airlin*
(m>1)	IC	→	*gyroscopic*	→	*gyroscop*
(m>1)	ABLE	→	*adjustable*	→	*adjust*
(m>1)	IBLE	→	*defensible*	→	*defens*
(m>1)	ANT	→	*irritant*	→	*irrit*
(m>1)	EMENT	→	*replacement*	→	*replac*
(m>1)	ENT	→	*dependent*	→	*depend*
(m>1)	and (*S or*T) ION	→	*adoption*	→	*adopt*
(m>1)	OU	→	*homologou*	→	*homolog*
(m>1)	ISM	→	*communism*	→	*commun*
(m>1)	ATE	→	*activate*	→	*activ*
(m>1)	ITI	→	*angulariti*	→	*angular*
(m>1)	OUS	→	*homologous*	→	*homolog*
(m>1)	IVE	→	*effective*	→	*effect*
(m>1)	IZE	→	*bowdlerize*	→	*bowdler*

Step 5a

(m>1)	E	→	*probate*	→	*probat*
(m=1 and not *o)	E	→	*cease*	→	*ceas*

Step 5b

(m>1 and *d and *L)	→	single letter	*controll*	→	*control*
			roll	→	*roll*